Sistema de Gestión de Inventarios basado en Arquitectura de Microservicios

Entrega 2: Diseño y arquitectura

Caso de estudio: Distribuidora El Montañero S.A.S.

Luis Alejandro Saraza Marín, Brayan David Collazos Escobedo y Mateo Falla Muñoz

Departamento de Informática y Computación, Facultad de Administración

Universidad Nacional de Colombia, Sede Manizales

Ingeniería de Software II

José Albeiro Montes Gil

25 de octubre de 2026

Resumen

Este documento presenta la segunda entrega del proyecto de aula de Ingeniería de Software II: el diseño del sistema de gestión de inventarios de Distribuidora El Montañero S.A.S., cuyo problema, alcance y requerimientos se definieron en la primera entrega. Primero se recogen las decisiones base: Laravel, Express y Flask repartidos entre un API Gateway propio y seis microservicios, una base de datos MySQL o PostgreSQL por componente y comunicación exclusivamente por REST. Después se presenta la arquitectura, con el Gateway como único punto de entrada, el descubrimiento de servicios por nombre y nueve llamadas entre componentes. Luego se describe el modelo de datos de cada componente con diagramas entidad-relación en notación de Chen, sin claves foráneas entre servicios. Por último, los contratos de API se documentan en tablas: convenciones comunes, enrutamiento del Gateway, matriz de permisos por rol y operaciones de cada componente. El documento cierra con la bitácora de uso de inteligencia artificial de la fase y las conclusiones.

Palabras clave: arquitectura de microservicios, API Gateway, contratos de API, modelo entidad-relación, gestión de inventarios.

Sistema de Gestión de Inventarios basado en Arquitectura de Microservicios

La primera entrega explicó por qué Distribuidora El Montañero, una empresa de Manizales con dos bodegas que lleva su inventario en Excel, necesita un sistema de gestión de inventarios y por qué conviene construirlo con microservicios (Saraza Marín et al., 2026). Allí se definieron el problema, el alcance, dieciocho requerimientos funcionales (RF01 a RF18), veintiún no funcionales (RNF01 a RNF21) y seis microservicios detrás de un API Gateway propio que también gestiona los usuarios. Esta segunda entrega convierte esa propuesta en un diseño que el equipo pueda construir en la siguiente fase.

Esta fase entrega el diagrama de arquitectura de microservicios, con sus componentes, la comunicación entre ellos, el API Gateway y el descubrimiento de servicios; el modelo de datos de cada microservicio; los contratos de API, y la bitácora de uso de IA. La contenerización con Docker, el despliegue en Kubernetes y las pruebas de rendimiento no se desarrollan en esta entrega. El código y la documentación del proyecto están en el repositorio del equipo (https://github.com/Mat-FM/Sistemas_Gestor_Inventario).

El documento se organiza así: el capítulo 1 reúne las decisiones base de tecnología y comunicación; el capítulo 2 presenta la arquitectura; el capítulo 3, el modelo de datos de cada componente; el capítulo 4, los contratos de API; el capítulo 5, la bitácora de uso de IA, y el capítulo 6, las conclusiones.

1. Decisiones base de diseño

1.1 Tecnología por componente

Solo se usan los frameworks y las bases de datos vistos en clase (RNF15): Laravel, Express y Flask, con MySQL y PostgreSQL. Se mantiene la asignación de la primera entrega: Laravel para el API Gateway, para los servicios de registro y consulta con reglas de negocio (catálogo y proveedores) y para el cliente web; Express para los servicios que atienden muchas peticiones al mismo tiempo (inventario y compras), y Flask para el procesamiento de datos y las tareas programadas (notificaciones y reportes). La Tabla 1 resume el framework, la base de datos y el puerto interno de cada componente.

Tabla 1

Tecnología, base de datos y puerto de cada componente

Componente	Framework	Base de datos	Puerto	Responsabilidad y requerimientos
API Gateway	Laravel	MySQL	8000	Único punto de entrada; usuarios, roles, inicio de sesión, permisos y límite de peticiones (RF07, RF08, RNF07, RNF09, RNF19).
ms-catalogo	Laravel	MySQL	8000	Productos, categorías y subcategorías; búsqueda y filtrado (RF01, RF02, RF12).
ms-proveedores	Laravel	MySQL	8000	Proveedores y productos que suministran (RF03).
ms-inventario	Express	PostgreSQL	3000	Bodegas, saldos, lotes, kardex, traslados y conteos (RF04, RF05, RF09, RF10, RF13, RF15, RF17, RF18).
ms-compras	Express	PostgreSQL	3000	Órdenes de compra, recepciones y devoluciones a proveedor (RF06, RF13).
ms-notificaciones	Flask	PostgreSQL	5000	Alertas en la interfaz y por correo; revisión diaria de vencimientos y de stock bajo (RF09, RF17).
ms-reportes	Flask	PostgreSQL	5000	Reportes, panel de indicadores, panel de auditoría e importación masiva (RF11, RF14, RF15, RF16).
Cliente web	Laravel	Ninguna	—	Interfaz adaptable para computador, tableta y celular, que solo se comunica con el Gateway (RNF21).

Nota. El puerto es el de cada servicio dentro del clúster; depende del framework: 8000 en Laravel, 3000 en Express y 5000 en Flask.

1.2 Comunicación entre servicios

Toda la comunicación usa peticiones REST con datos en formato JSON (RNF01), como se decidió en la primera entrega; no se usa un intermediario de mensajería. Las peticiones externas siempre entran por el Gateway, y entre servicios solo se hacen las nueve llamadas de la Tabla 2. Se distinguen dos tipos de llamada:

- **Llamadas que esperan respuesta.** El servicio necesita el resultado para continuar; por ejemplo, ms-inventario confirma en ms-catalogo que el producto existe antes de registrar un movimiento. Si el otro servicio no responde, la operación se rechaza con el código 503.
- **Avisos que no detienen la operación.** El servicio primero guarda su operación y después avisa a otro servicio. Si el destino no responde, la operación ya quedó guardada y el fallo se anota en el registro del servicio (RNF10). Así, un fallo en notificaciones o en reportes no impide registrar movimientos.

En los dos casos, cada llamada espera como máximo 3 segundos. Solo se reintentan las operaciones que no se duplican al repetirse, hasta tres veces, con esperas de 0,2, 0,4 y 0,8 segundos, y si un servicio falla cinco veces seguidas se deja de llamar durante 30 segundos antes de volver a probar (RNF18). Las operaciones que registran movimientos llevan una clave única para que un reintento no las duplique (RNF17).

1.3 Documentación de los contratos

Los contratos de API se documentan en tablas (capítulo 4): para cada operación se indica el método, la ruta, los roles autorizados, los datos de entrada, la respuesta y los errores posibles. Esta forma permite que cada integrante construya su servicio sobre una interfaz acordada, sin depender de cómo programe los demás. Cuando los servicios estén implementados, cada uno publicará además su especificación OpenAPI en la ruta /docs (RNF08).

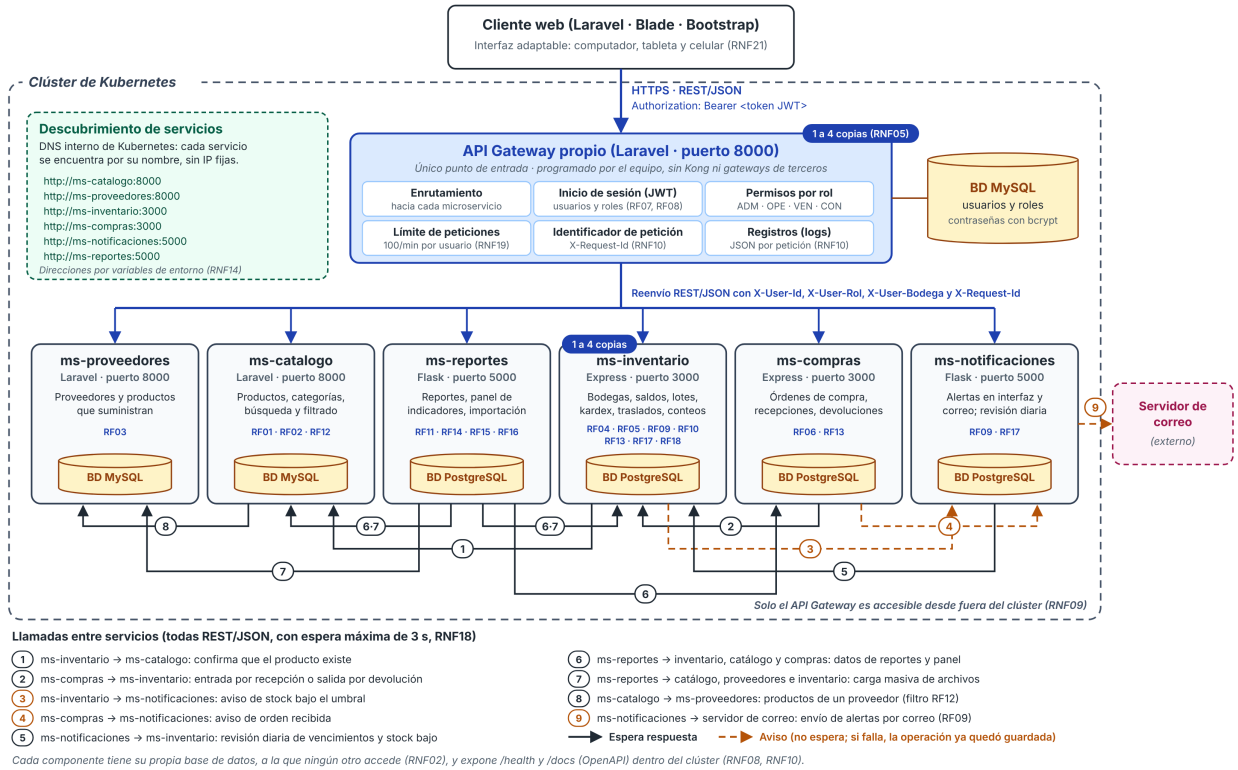
2. Arquitectura de microservicios

2.1 Vista general

La Figura 1 muestra la arquitectura del sistema. El cliente web se comunica solo con el API Gateway, por HTTPS y con el token de sesión en la cabecera Authorization. El Gateway verifica al usuario con su propia base de datos, revisa su rol y reenvía la petición por REST al microservicio que corresponde. Cada uno de los seis microservicios tiene su propia base de datos, a la que ningún otro componente accede (RNF02; Richardson, s. f.-b), y todos se ejecutan dentro de un clúster de Kubernetes, del que solo el Gateway es accesible desde fuera (RNF09).

Figura 1

Diagrama de arquitectura de microservicios



Nota. Las flechas azules son las peticiones que el Gateway reenvía. Las líneas numeradas son las llamadas entre servicios de la Tabla 2: las continuas esperan respuesta y las punteadas naranjas son avisos. «1 a 4 copias» indica los dos componentes que aumentan su número de copias cuando hay más demanda (RNF05). Elaboración propia.

En la parte superior está el cliente web, construido en Laravel con plantillas Blade y Bootstrap. Dentro del clúster están el API Gateway con su base de datos de usuarios y roles, el recuadro de descubrimiento de servicios y las seis tarjetas de los microservicios, cada una con su framework, su puerto, su responsabilidad, los requerimientos que atiende y su base de datos. Fuera del clúster, a la derecha, está el servidor de correo externo que usa ms-notificaciones para enviar las alertas.

2.2 API Gateway

El Gateway es el único punto de entrada del sistema y aplica el patrón API Gateway (Richardson, s. f.-a). Lo programa el equipo en Laravel, sin Kong ni otros gateways de terceros (RNF09), y concentra las funciones que comparten todos los servicios:

- **Enrutamiento:** dirige cada petición al microservicio que corresponde, según su ruta (Tabla 4).
- **Inicio de sesión:** gestiona los usuarios y los roles (RF07, RF08) y entrega un token de sesión JWT, firmado y válido por 60 minutos (Jones et al., 2015). Las contraseñas se guardan cifradas con bcrypt (RNF07).
- **Permisos por rol:** revisa en cada petición que el rol del usuario tenga permiso para la operación, según la matriz de la Tabla 5. Si no hay token o no es válido responde 401, y si el rol no tiene permiso, 403.
- **Límite de peticiones:** acepta 100 peticiones por minuto por usuario y 10 intentos de inicio de sesión por minuto desde una misma dirección IP; al superar el límite responde con el código 429 y la cabecera Retry-After, que indica cuándo se puede volver a intentar (Nottingham y Fielding, 2012; RNF19).
- **Identificador de petición y registros:** asigna a cada petición un identificador único y escribe un registro en formato JSON por cada una (RNF10).

Al reenviar una petición a un microservicio, el Gateway agrega estas cabeceras:

- **X-User-Id:** identificador del usuario ya verificado. Los servicios lo guardan para saber quién hizo cada operación (RF10).

- **X-User-Rol:** rol del usuario (ADMINISTRADOR, OPERADOR_BODEGA, VENDEDOR o CONTADOR).
- **X-User-Bodega:** bodega asignada, cuando el usuario es operador de bodega. Con ella, ms-inventario y ms-compras limitan sus operaciones a esa bodega.
- **X-Request-Id:** identificador de la petición. Viaja también en las llamadas entre servicios, para seguir una misma operación en los registros de todos los componentes (RNF10).

Las reglas del inventario no viven en el Gateway, sino en cada microservicio. Como el Gateway recibe todo el tráfico, es uno de los dos componentes, junto con ms-inventario, que pasan automáticamente de una a cuatro copias cuando hay más demanda (RNF05).

2.3 Descubrimiento de servicios

Los servicios se encuentran entre sí por su nombre, con el DNS interno de Kubernetes, que asigna un nombre a cada servicio del clúster (Kubernetes, 2025). Así no se necesitan direcciones IP fijas ni una herramienta adicional de descubrimiento de servicios. Las direcciones que usan los componentes son http://ms-catalogo:8000, http://ms-proveedores:8000, http://ms-inventario:3000, http://ms-compras:3000, http://ms-notificaciones:5000 y http://ms-reportes:5000. Cada componente las recibe en variables de entorno y no las tiene escritas en su código (RNF14). La configuración del clúster no hace parte de esta entrega.

2.4 Llamadas entre servicios

La Tabla 2 detalla las llamadas entre servicios, con la misma numeración de la Figura 1, y la operación de cada contrato que se usa en ellas.

Tabla 2

Llamadas entre servicios

N.°	Origen → destino	Para qué	Tipo	Operación que se usa
1	ms-inventario → ms-catalogo	Confirmar que el producto existe	Espera respuesta	GET /productos/{id}
2	ms-compras → ms-inventario	Registrar la entrada por recepción o la salida por devolución	Espera respuesta	POST /movimientos (ENTRADA_COMPRA o DEVOLUCION_PROVEEDOR)
3	ms-inventario → ms-notificaciones	Avisar que un producto quedó bajo el umbral	Aviso	POST /internas/notificaciones (STOCK_BAJO)
4	ms-compras → ms-notificaciones	Avisar que una orden se recibió	Aviso	POST /internas/notificaciones (ORDEN_RECIBIDA)
5	ms-notificaciones → ms-inventario	Revisión diaria de vencimientos y de stock bajo	Espera respuesta	GET /lotes y GET /saldos/bajo-umbral
6	ms-reportes → ms-inventario, ms-catalogo y ms-compras	Obtener los datos de los reportes y del panel de indicadores	Espera respuesta	GET /saldos, /saldos/bajo-umbral, /movimientos, /lotes y /bodegas; GET /productos y /categorias; GET /ordenes-compra
7	ms-reportes → ms-catalogo, ms-proveedores y ms-inventario	Enviar las filas válidas de la carga masiva	Espera respuesta	POST /internas/productos/lote, /internas/proveedores/lote y /internas/saldos/carga-inicial
8	ms-catalogo → ms-proveedores	Obtener los productos de un proveedor (filtro de RF12)	Espera respuesta	GET /proveedores/{id}/productos
9	ms-notificaciones → servidor de correo	Enviar las alertas por correo (RF09)	Aviso	Envío del correo al servidor externo

Nota. Todas las llamadas usan REST con JSON y esperan como máximo 3 segundos (RNF18). En los avisos, si el destino no responde, la operación de origen ya quedó guardada. Elaboración propia.

3. Modelo de datos por microservicio

3.1 Decisiones de modelado

- **Base de datos por servicio** (RNF02): cada componente, incluido el Gateway, es dueño de su base de datos, y ningún otro servicio la lee ni la modifica (Richardson, s. f.-b).
- **Sin claves foráneas entre servicios.** Cuando un servicio se refiere a un dato de otro, guarda solo su identificador (por ejemplo, id_producto o id_usuario) y lo valida por medio de la API del servicio dueño. En los diagramas, estos atributos llevan la aclaración «ref. a» seguida del servicio dueño.
- **Dar de baja en lugar de borrar.** Los datos que tienen historia, como productos, proveedores, bodegas y usuarios, se marcan como inactivos y no se eliminan (RF01).
- **Los reportes no se guardan.** ms-reportes calcula los reportes y los indicadores en el momento, con datos de los demás servicios; su base de datos solo guarda las importaciones masivas (RF16).
- **Nombres.** Los atributos se escriben como se llamarán las columnas, en minúsculas, sin tildes y con guion bajo (por ejemplo, id_categoria o descripcion).

Respecto a la primera entrega se agregaron cinco entidades que el diseño detallado mostró necesarias: DetalleConteo y UmbralProducto en ms-inventario, DevolucionProveedor en ms-compras, Configuracion en ms-notificaciones y ErrorImportacion en ms-reportes. La Tabla 3 resume las entidades de cada componente.

Tabla 3

Entidades de cada componente

Componente	Base de datos	Entidades	Agregadas en esta entrega
API Gateway	MySQL	Rol, Usuario	—
ms-catalogo	MySQL	Categoria, Producto	—
ms-proveedores	MySQL	Proveedor, ProveedorProducto	—
ms-inventario	PostgreSQL	Bodega, Saldo, Lote, Movimiento, Traslado, Conteo, DetalleConteo, UmbralProducto	DetalleConteo, UmbralProducto
ms-compras	PostgreSQL	OrdenCompra, DetalleOrden, Recepcion, DevolucionProveedor	DevolucionProveedor
ms-notificaciones	PostgreSQL	Notificacion, Preferencia, Configuracion	Configuracion
ms-reportes	PostgreSQL	Importacion, ErrorImportacion	ErrorImportacion

Nota. Elaboración propia.

3.2 Notación

Los diagramas usan la notación entidad-relación de Chen (1976), con estas convenciones:

- Entidad: rectángulo. Relación: rombo. Atributo: óvalo.
- Atributo clave: nombre subrayado. Bajo el nombre de algunos atributos aparece, en letra pequeña, una aclaración: los valores posibles, si el dato es único o a qué servicio hace referencia.
- Participación total: línea doble entre la entidad y la relación; indica que toda instancia de la entidad participa en la relación. Participación parcial: línea simple.
- Cardinalidad: 1 y N junto a cada línea.

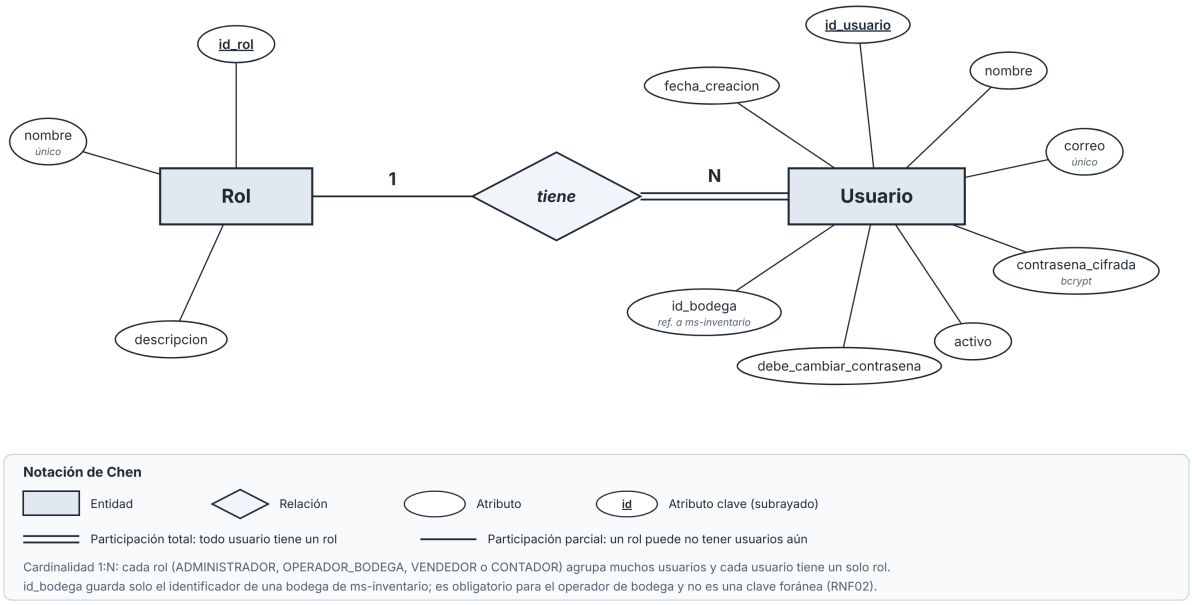
Las claves internas que unen dos entidades del mismo servicio no se dibujan como atributos, porque la relación ya las representa; las referencias a datos de otro servicio sí se dibujan, con la aclaración «ref.». Cada diagrama incluye una leyenda con estos símbolos. Los diagramas muestran el modelo conceptual; las tablas físicas se definirán al implementar cada servicio.

3.3 API Gateway

El Gateway guarda la identidad del sistema (Figura 2). Cada usuario tiene un solo rol, y cada rol agrupa varios usuarios (relación «tiene», 1:N); la línea doble indica que todo usuario debe tener un rol, mientras que un rol puede no tener usuarios todavía. Los roles son los cuatro de la primera entrega: ADMINISTRADOR, OPERADOR_BODEGA, VENDEDOR y CONTADOR. El correo es único y con él se inicia sesión; la contraseña se guarda cifrada con bcrypt; activo permite desactivar a un usuario sin borrarlo, y debe_cambiar_contrasena queda activo cuando el administrador asigna una contraseña temporal (RF08). id_bodega guarda la bodega del operador de bodega, que es un dato de ms-inventario.

Figura 2

Modelo entidad-relación del API Gateway



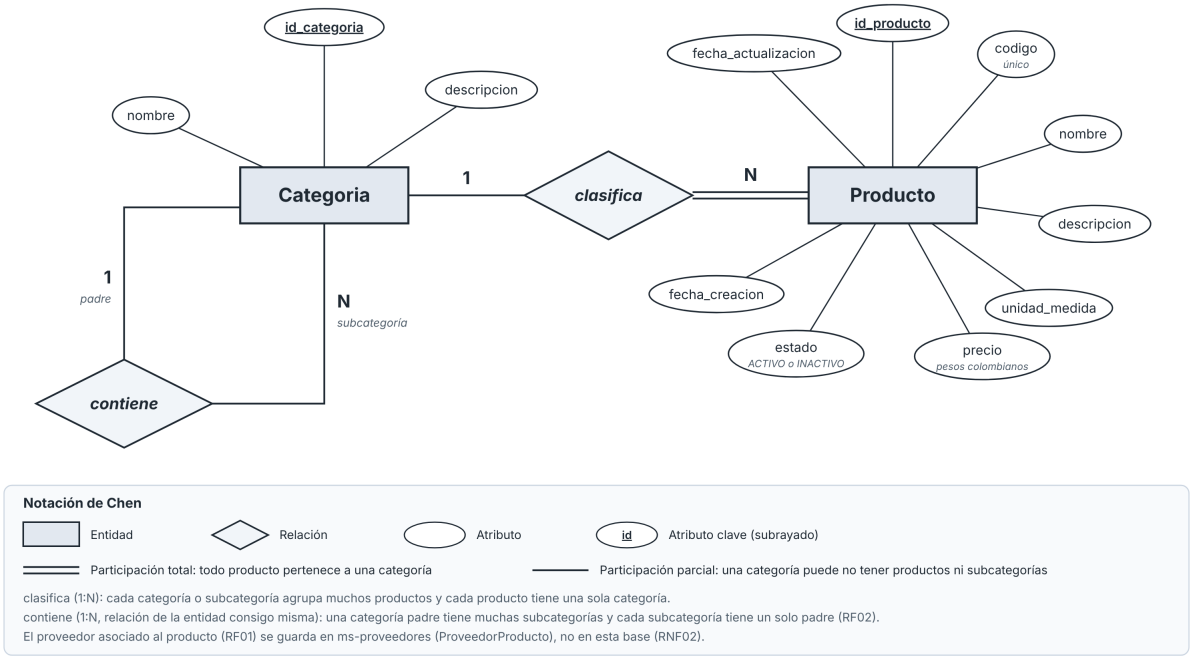
Nota. Elaboración propia con notación de Chen (1976).

3.4 ms-catalogo

Categoria clasifica muchos productos y cada producto tiene una sola categoría (relación «clasifica», 1:N, con participación total del producto). Categoria se relaciona consigo misma mediante «contiene» para formar las subcategorías de RF02: una categoría padre tiene muchas subcategorías y cada subcategoría tiene un solo padre; las dos líneas son simples, porque una categoría principal no tiene padre y puede no tener subcategorías (Figura 3). El código del producto es único; precio se registra en pesos colombianos y se usa en la valorización (RF11), y estado (ACTIVO o INACTIVO) permite dar de baja un producto sin borrarlo. El proveedor asociado de RF01 no se guarda aquí, sino en ms-proveedores.

Figura 3

Modelo entidad-relación de ms-catalogo



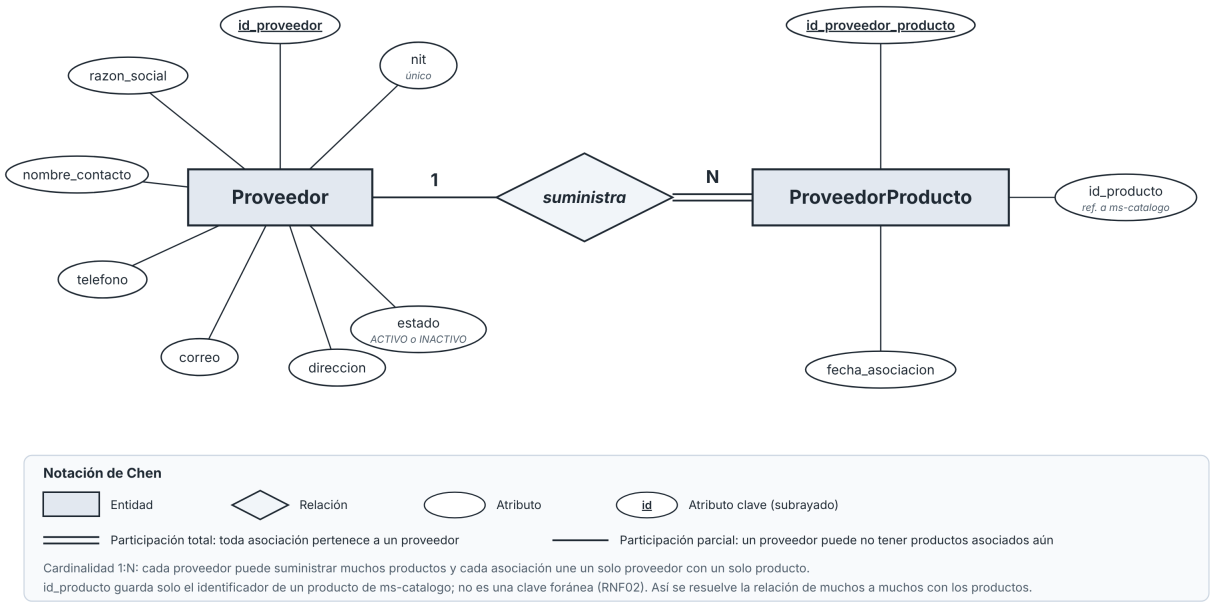
Nota. Elaboración propia con notación de Chen (1976).

3.5 ms-proveedores

Proveedor guarda los datos de la empresa y de su contacto; el NIT es único y estado (ACTIVO o INACTIVO) permite darlo de baja (Figura 4). La relación de muchos a muchos entre proveedores y productos se resuelve con la entidad ProveedorProducto, porque los productos viven en ms-catalogo: cada proveedor suministra muchos productos, y cada asociación une un solo proveedor con un solo producto (relación «suministra», 1:N, con participación total de la asociación). id_producto es una referencia a ms-catalogo.

Figura 4

Modelo entidad-relación de ms-proveedores



Nota. Elaboración propia con notación de Chen (1976).

3.6 ms-inventario

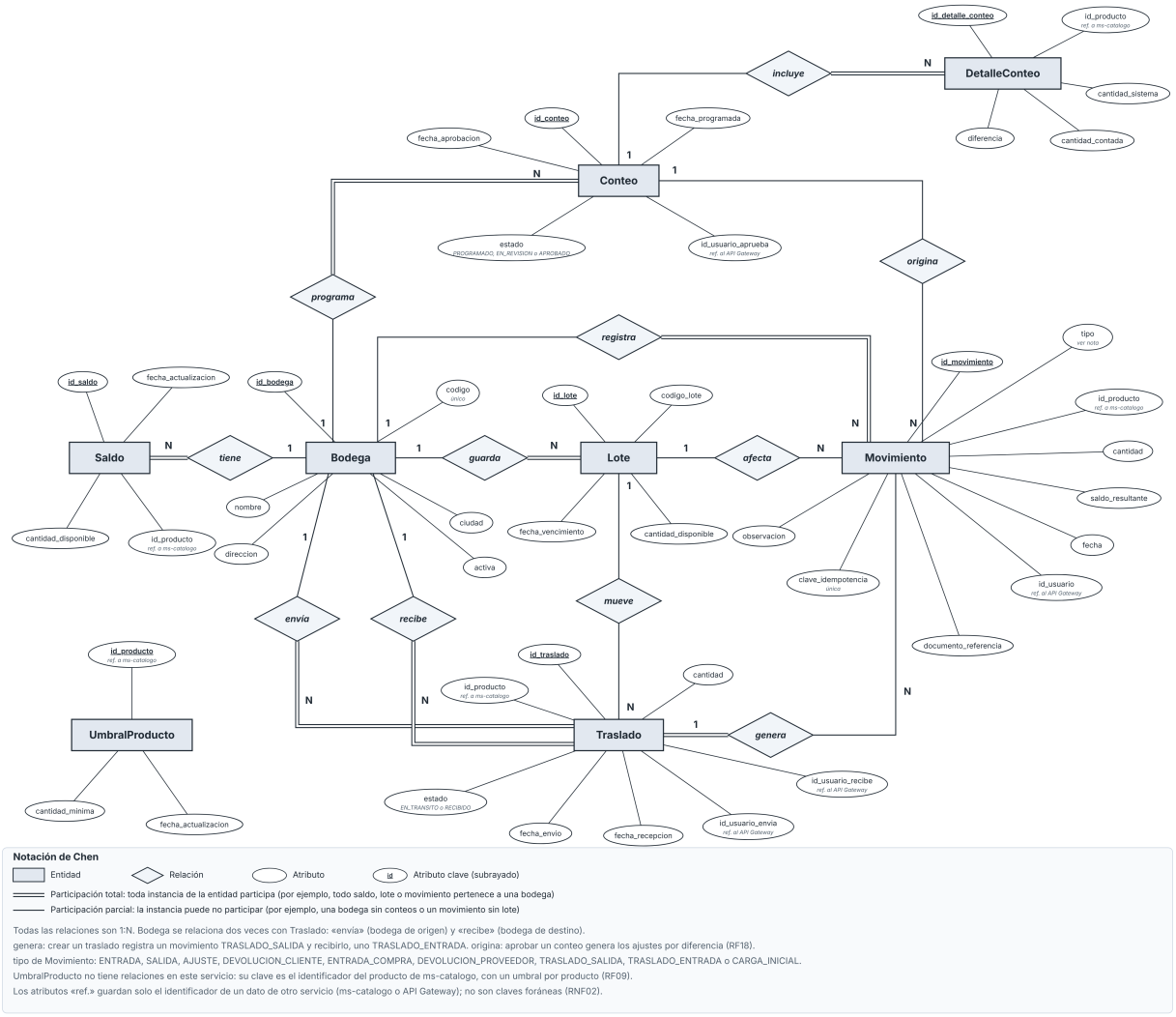
Es el modelo más grande, con ocho entidades y once relaciones, todas 1:N (Figura 5). Bodega es el centro: tiene los saldos de cada producto («tiene»), guarda los lotes («guarda»), registra los movimientos («registra») y programa los conteos («programa»). Bodega se relaciona dos veces con Traslado: como bodega de origen («envía») y como bodega de destino («recibe»). Movimiento es el kardex (RF10): guarda el tipo, la cantidad, el saldo resultante, la fecha, quién lo hizo y una clave de idempotencia única que evita duplicados (RNF17).

Un movimiento puede venir de un lote («afecta»), de un traslado («genera») o de un conteo («origina»); por eso esas tres relaciones son parciales del lado de Movimiento. Crear un traslado registra un movimiento TRASLADO_SALIDA y recibirlo, uno TRASLADO_ ENTRADA, y aprobar un conteo genera los ajustes por diferencia (RF18). Un traslado puede llevar un lote («mueve»), y cada conteo incluye sus líneas en DetalleConteo, con la cantidad del sistema, la contada y la diferencia. UmbralProducto no tiene relaciones: su clave es el identificador del producto de ms-catalogo, con un umbral por producto; si la suma de las dos bodegas queda por debajo de cantidad_minima, se genera la alerta (RF09).

Hay nueve tipos de movimiento: ENTRADA, SALIDA, AJUSTE, DEVOLUCION_ CLIENTE, ENTRADA_COMPRA, DEVOLUCION_PROVEEDOR, TRASLADO_SALIDA, TRASLADO_ENTRADA y CARGA_INICIAL. La devolución de un cliente (RF13) es un movimiento DEVOLUCION_CLIENTE, con la factura o la remisión como documento de referencia.

Figura 5

Modelo entidad-relación de ms-inventario



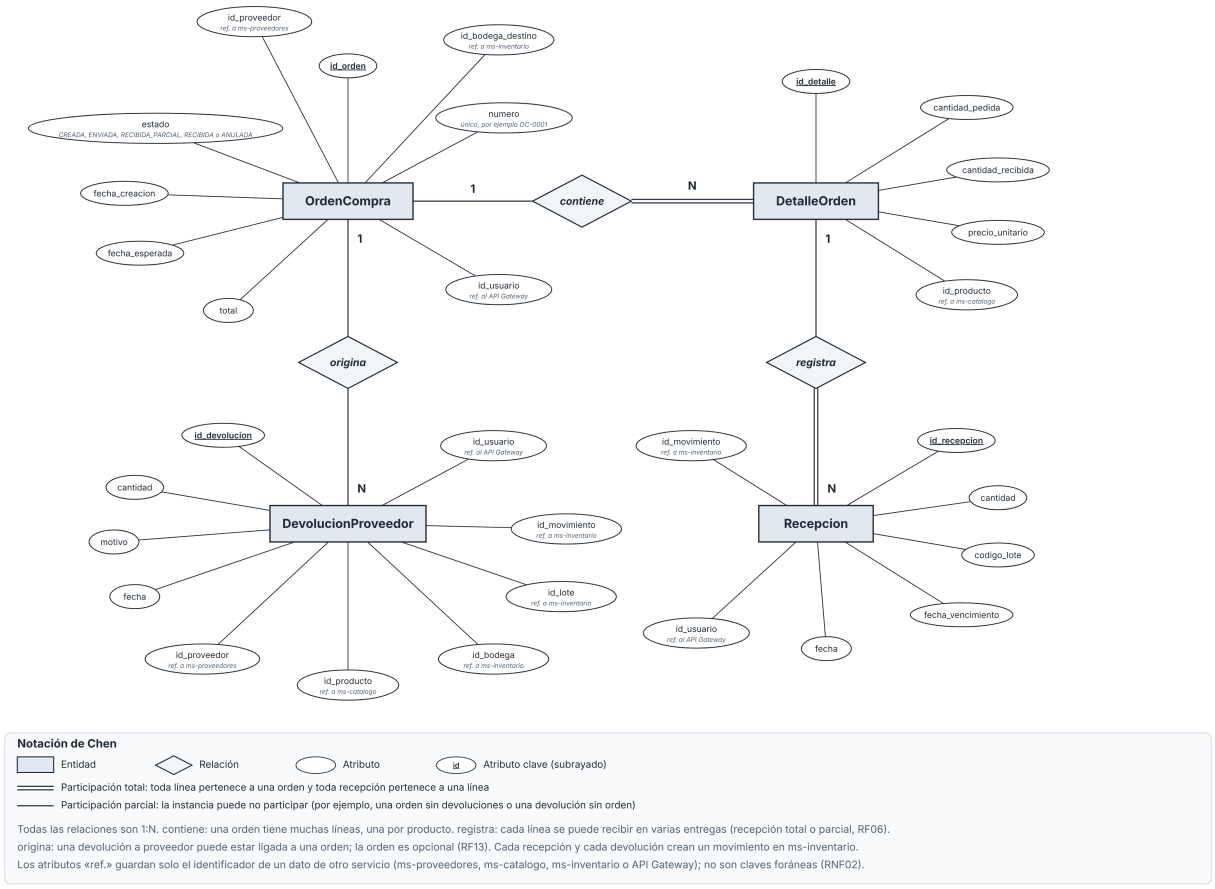
Nota. Elaboración propia con notación de Chen (1976).

3.7 ms-compras

OrdenCompra contiene muchas líneas en DetalleOrden, una por producto («contiene»), y cada línea se puede recibir en varias entregas, lo que permite la recepción total o parcial de RF06 («registra»); las líneas y las recepciones tienen participación total (Figura 6). Los estados de la orden son CREADA, ENVIADA, RECIBIDA_PARCIAL, RECIBIDA y ANULADA, y el número es un consecutivo único, por ejemplo OC-0001. DevolucionProveedor registra las devoluciones a proveedor de RF13; puede estar ligada a una orden, pero la orden es opcional, por eso «origina» es parcial en los dos lados. Cada recepción y cada devolución guardan el identificador del movimiento que crearon en ms-inventario.

Figura 6

Modelo entidad-relación de ms-compras



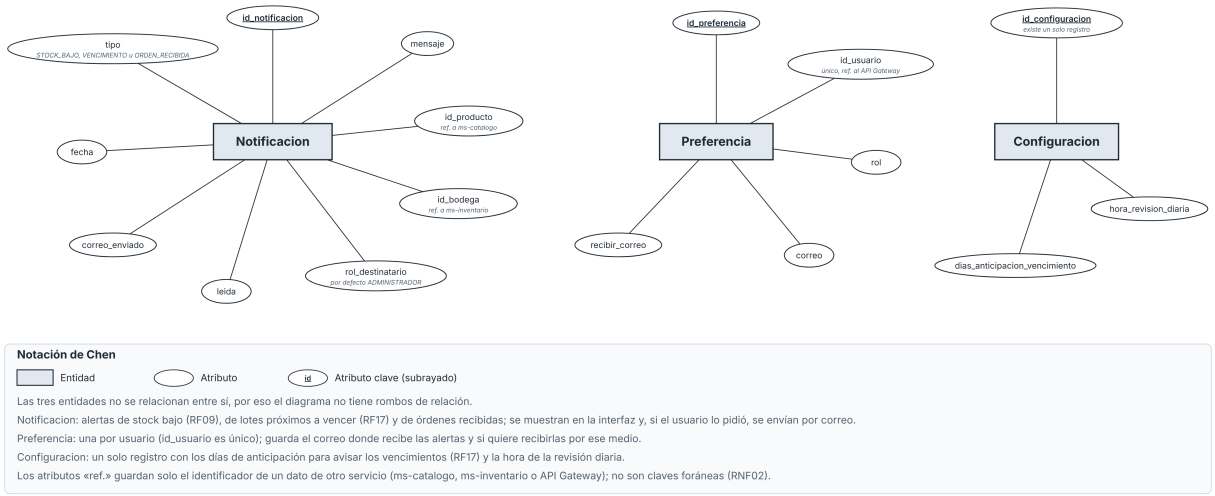
Nota. Elaboración propia con notación de Chen (1976).

3.8 ms-notificaciones

Las tres entidades de este servicio no se relacionan entre sí, por eso el diagrama no tiene rombos (Figura 7). Notificacion guarda las alertas de stock bajo (RF09), de lotes próximos a vencer (RF17) y de órdenes recibidas; cada una va dirigida a un rol, ADMINISTRADOR por defecto, y registra si ya se leyó y si se envió por correo. Preferencia guarda, para cada usuario, el correo donde recibe las alertas y si quiere recibirlas por ese medio. Configuracion tiene un solo registro con los días de anticipación para avisar los vencimientos (RF17) y la hora de la revisión diaria.

Figura 7

Modelo entidad-relación de ms-notificaciones



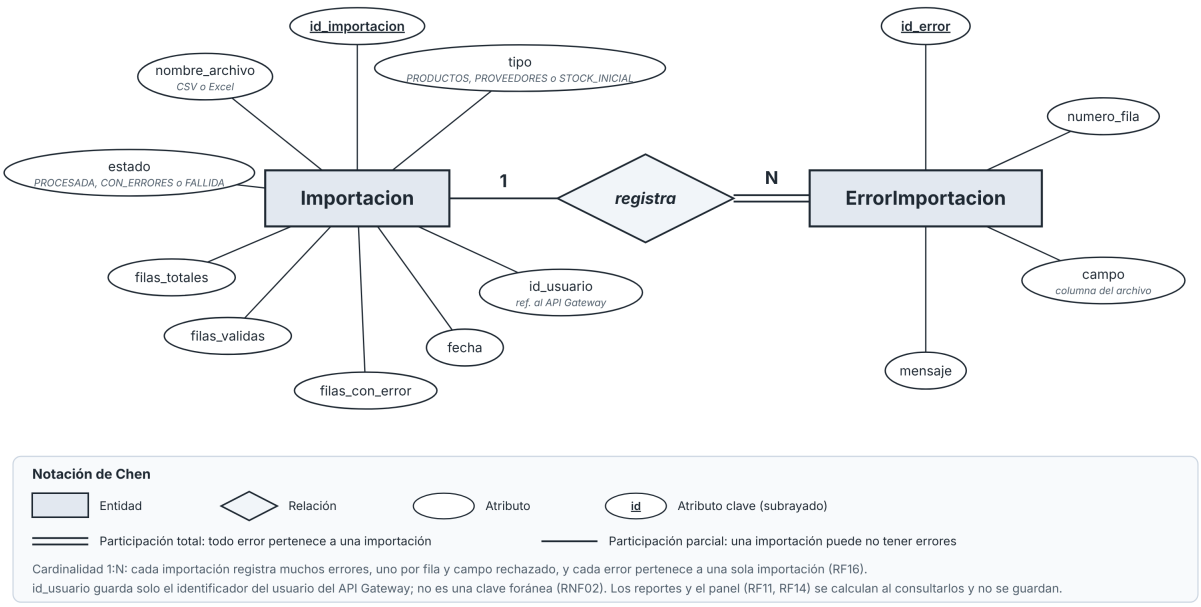
Nota. Elaboración propia con notación de Chen (1976).

3.9 ms-reportes

ms-reportes solo guarda las importaciones masivas de RF16 (Figura 8). Importacion registra el tipo de archivo (PRODUCTOS, PROVEEDORES o STOCK_INICIAL), su nombre, el estado (PROCESADA, CON_ERRORES o FALLIDA) y cuántas filas se leyeron, cuántas fueron válidas y cuántas se rechazaron. Cada importación registra sus errores en ErrorImportacion, uno por fila y campo rechazado, con lo que se cumple el reporte de errores por fila de RF16 (relación «registra», 1:N). Los reportes y el panel de indicadores (RF11, RF14) se calculan al consultarlos y no se guardan.

Figura 8

Modelo entidad-relación de ms-reportes



Nota. Elaboración propia con notación de Chen (1976).

4. Contratos de API

Este capítulo presenta las convenciones comunes a todas las API, el enrutamiento del Gateway, la matriz de permisos por rol y las operaciones de cada componente. Los nombres de los campos son los mismos atributos de los modelos de datos del capítulo 3.

4.1 Convenciones comunes

- **Rutas.** Las rutas públicas están en el Gateway con el prefijo /api (por ejemplo, GET /api/productos/{id}), y el Gateway las reenvía al servicio sin ese prefijo (GET /productos/{id}). Las tablas de las secciones 4.5 a 4.10 muestran la ruta interna.
- **Autenticación.** Toda ruta pública exige la cabecera Authorization: Bearer <token JWT>, salvo el inicio de sesión. Hacia los servicios viajan las cabeceras de identidad de la sección 2.2.
- **Formato.** Los datos van en JSON con codificación UTF-8, y las fechas en formato ISO 8601 (por ejemplo, 2026-10-25T14:32:05Z).
- **Filtros y campos opcionales.** En la columna Entrada, ?a&b indica los parámetros de consulta que acepta la ruta, los filtros que se agregan al final de la dirección (por ejemplo, GET /api/saldos?id_producto=25&id_bodega=1); todos son opcionales, salvo que la tabla diga lo contrario. En los cuerpos JSON, un campo terminado en ? es opcional; por ejemplo, en {nombre, descripcion?} la descripción se puede omitir.
- **Listados.** Se paginan con ?pagina=1&por_pagina=20 (máximo 100) y responden {"datos": [...], "meta": {"pagina", "por_pagina", "total"}}.
- **Errores.** Siempre con el cuerpo {"error": {"codigo", "mensaje", "detalles"}} y los códigos 400 (petición mal formada), 401 (sin autenticar), 403 (sin permiso), 404 (no existe), 409 (conflicto: dato duplicado, cambio de estado no permitido o saldo insuficiente), 422 (datos no válidos), 429 (demasiadas peticiones, RNF19) y 503 (un servicio del que depende no responde).
- **Idempotencia.** Las operaciones que registran movimientos llevan la cabecera Idempotency-Key con una clave única. Si la misma petición llega dos veces, el

servicio devuelve la respuesta original y no repite la operación (RNF17).

- **Dar de baja en lugar de borrar.** Los datos con historia se inactivan con POST /<recurso>/{id}/inactivar y se reactivan con /activar.
- **Rutas internas.** Las rutas que empiezan por /internas solo las llaman otros servicios dentro del clúster; el Gateway no las publica.
- **Salud y documentación.** Cada servicio expone GET /health para comprobar que está activo (RNF10) y GET /docs con su documentación (RNF08); estas rutas no pasan por el Gateway.

4.2 Enrutamiento del Gateway

La Tabla 4 muestra a qué componente reenvía el Gateway cada grupo de rutas públicas.

Tabla 4

Enrutamiento de las rutas públicas en el API Gateway

Prefijo público	Destino	Ruta interna
/api/auth/, */api/usuarios/*, /api/roles	API Gateway	Se atienden en el propio Gateway.
/api/productos/, */api/categorias/*	ms-catalogo	La misma ruta, sin /api.
/api/proveedores/*	ms-proveedores	La misma ruta, sin /api.
/api/bodegas/, */api/saldos/*, /api/umbrales/, */api/movimientos/*, /api/lotes/, */api/traslados/*, /api/conteos/*	ms-inventario	La misma ruta, sin /api.
/api/ordenes-compra/, */api/devoluciones-proveedor/*	ms-compras	La misma ruta, sin /api.
/api/notificaciones/, */api/preferencias/*, /api/configuracion-alertas	ms-notificaciones	La misma ruta, sin /api.
/api/reportes/, */api/indicadores, /api/importaciones/*	ms-reportes	La misma ruta, sin /api.

Nota. El asterisco (*) representa cualquier ruta que siga al prefijo. Elaboración propia.

4.3 Matriz de permisos por rol

La Tabla 5 traduce los cuatro roles de la primera entrega en los permisos que aplica el Gateway. «Ver» significa solo consulta (GET) y «Su bodega», que el operador solo opera sobre la bodega que tiene asignada. Se agregaron tres precisiones necesarias para operar: el operador de bodega puede consultar los proveedores y las órdenes de compra para registrar recepciones y devoluciones; también puede consultar el kardex de su bodega para revisar los movimientos que registra, y todos los roles reciben las notificaciones dirigidas a su rol.

Tabla 5

Matriz de permisos por rol

Operación	Administrador	Operador de bodega	Vendedor	Contador
Usuarios, roles y restablecimiento de contraseñas	Sí	—	—	—
Productos y categorías	Sí	Ver	Ver	Ver
Proveedores y sus productos	Sí	Ver	—	—
Bodegas	Sí	Ver	Ver	Ver
Consultar existencias por bodega	Sí	Sí	Sí	Sí
Umbrales de stock	Sí	—	—	—
Entradas, salidas, ajustes y devoluciones de cliente	Sí	Su bodega	—	—
Lotes y sugerencia de salida	Sí	Su bodega	—	—
Traslados	Sí	Su bodega	—	—
Kardex	Sí	Su bodega	—	Ver
Programar y aprobar conteos físicos	Sí	—	—	—
Registrar cantidades contadas	Sí	Su bodega	—	—
Crear órdenes de compra y cambiar su estado	Sí	Ver	—	—
Recibir mercancía y devolver a proveedor	Sí	Su bodega	—	—
Notificaciones de su rol y preferencia de correo	Sí	Sí	Sí	Sí
Configuración de alertas	Sí	—	—	—
Reportes, panel de indicadores y panel de auditoría	Sí	—	—	Sí
Importación masiva	Sí	—	—	—

Nota. Elaboración propia con base en los roles de la primera entrega (Saraza Marín et al., 2026).

En la columna Roles, ADM es el administrador, OPE el operador de bodega, VEN el vendedor y CON el contador; Todos son los cuatro roles, Autenticado es cualquier usuario con sesión iniciada e interno indica que otro servicio llama la ruta dentro del clúster.

4.4 API Gateway (Laravel)

El Gateway atiende estas rutas con su propia base de datos de usuarios; no las reenvía a ningún microservicio. Por eso la tabla muestra la ruta pública, con el prefijo /api.

Tabla 6

Operaciones de API Gateway

Método	Ruta pública	Roles	Entrada	Respuesta	Errores
POST	/api/auth/login	Público	{correo, contrasena}	200 {token, expira_en, usuario: {id_usuario, nombre, rol, id_bodega, debe_cambiar_ contrasena}}	401 credenciales inválidas; 403 usuario inactivo; 422; 429 más de 10 intentos por minuto (RNF19)
POST	/api/auth/logout	Autenticado	—	204 (RF08)	401
GET	/api/auth/me	Autenticado	—	200 usuario con su rol	401
PUT	/api/auth/contrasena	Autenticado	{contrasena_actual, contrasena_nueva}	204; debe_cambiar_ contrasena pasa a falso (RF08)	401 contraseña actual incorrecta; 422 contraseña nueva no válida
GET	/api/usuarios	ADM	?id_rol&activo&pagina& por_pagina	200 listado paginado, sin contraseñas	—
GET	/api/usuarios/{id}	ADM	—	200 usuario	404
POST	/api/usuarios	ADM	{nombre, correo, id_rol, id_bodega?, contrasena_ temporal}	201 usuario con debe_ cambiar_contrasena verdadero (RF07)	404 rol inexistente; 409 CORREO_ DUPLICADO; 422 operador sin bodega
PUT	/api/usuarios/{id}	ADM	{nombre, correo, id_rol, id_bodega?}	200 usuario	404; 409 CORREO_ DUPLICADO; 422 operador sin bodega
POST	/api/usuarios/ {id}/inactivar	ADM	—	200; el usuario ya no puede iniciar sesión	404
POST	/api/usuarios/{id}/activar	ADM	—	200	404
POST	/api/usuarios/ {id}/restablecer-contrasena	ADM	{contrasena_temporal}	204; el usuario debe cambiarla en su siguiente inicio de sesión (RF08)	404; 422
GET	/api/roles	ADM	—	200 los cuatro roles	—

El restablecimiento reemplaza la recuperación de contraseña por correo (RF08): el administrador asigna una contraseña temporal y, mientras debe_cambiar_contrasena sea verdadero, el Gateway solo permite usar PUT /api/auth/contrasena y POST /api/auth/logout.

Al cerrar sesión, el cliente web borra el token, que de todas formas deja de servir al

cumplir sus 60 minutos (RNF07). Los usuarios no se borran: se inactivan. Al crear un operador de bodega, el administrador elige la bodega de la lista que el cliente web obtiene de GET /api/bodegas.

4.5 ms-catalogo (Laravel)

Tabla 7

Operaciones de ms-catalogo

Método	Ruta interna	Roles	Entrada	Respuesta	Errores
GET	/productos	Todos; ms-reportes (interno)	?nombre&codigo&id_ categoria&id_ proveedor&estado& pagina&por_pagina	200 listado paginado (RF12)	422 filtros no válidos; 503 ms-proveedores no responde (solo al filtrar por proveedor)
GET	/productos/{id}	Todos; ms-inventario (interno)	—	200 producto	404
POST	/productos	ADM	{codigo, nombre, descripcion?, id_ categoria, unidad_ medida, precio}	201 producto en estado ACTIVO (RF01)	404 categoría inexistente; 409 CODIGO_ DUPLICADO; 422
PUT	/productos/{id}	ADM	Igual que POST, sin codigo	200 producto	404; 422
POST	/productos/{id}/inactivar	ADM	—	200; el producto pasa a INACTIVO (dar de baja, RF01)	404
POST	/productos/{id}/activar	ADM	—	200	404
GET	/categorias	Todos; ms-reportes (interno)	?id_categoria_padre& arbol	200 categorías; con arbol=true, cada una con sus subcategorías	—
GET	/categorias/{id}	Todos	—	200 categoría	404
POST	/categorias	ADM	{nombre, descripcion?, id_categoria_padre?}	201 categoría o subcategoría (RF02)	404 categoría padre inexistente; 409 nombre repetido en el mismo nivel; 422
PUT	/categorias/{id}	ADM	Igual que POST	200 categoría	404; 409; 422 una categoría no puede ser su propia padre
DELETE	/categorias/{id}	ADM	—	204 (RF02)	404; 409 CATEGORIA_EN_ USO (tiene productos o subcategorías)
POST	/internas/productos/lote	ms-reportes (interno)	{filas: [{codigo, nombre, descripcion, id_ categoria, unidad_ medida, precio}]}	200 {creados, rechazados: [{fila, motivo}]} (RF16)	422 lista vacía

Al filtrar por id_proveedor, ms-catalogo pide a ms-proveedores los productos de ese proveedor con GET /proveedores/{id}/productos (llamada 8) y devuelve solo esos. El proveedor

asociado de RF01 se registra en ms-proveedores con POST /proveedores/{id}/productos, desde el mismo formulario del cliente web.

Los productos no se borran, porque su historial sigue en el kardex; se dan de baja con /inactivar. Las categorías sí se pueden eliminar, pero solo cuando no tienen productos ni subcategorías.

4.6 ms-proveedores (Laravel)

Tabla 8

Operaciones de ms-proveedores

Método	Ruta interna	Roles	Entrada	Respuesta	Errores
GET	/proveedores	ADM, OPE	?nit&razon_social&id_producto&estado&pagina&por_pagina	200 listado paginado	—
GET	/proveedores/{id}	ADM, OPE	—	200 proveedor	404
POST	/proveedores	ADM	{nit, razon_social, nombre_contacto, telefono, correo, direccion}	201 proveedor en estado ACTIVO (RF03)	409 NIT_DUPLICADO; 422
PUT	/proveedores/{id}	ADM	Igual que POST, sin nit	200 proveedor	404; 422
POST	/proveedores/{id}/inactivar	ADM	—	200; el proveedor pasa a INACTIVO (RF03)	404
POST	/proveedores/{id}/activar	ADM	—	200	404
GET	/proveedores/{id}/productos	ADM, OPE; ms-catalogo (interno)	—	200 [{id_proveedor_producto, id_producto, fecha_asociacion}]	404
POST	/proveedores/{id}/productos	ADM	{id_producto}	201 asociación (RF01, RF03)	404; 409 el producto ya está asociado
DELETE	/proveedores/{id}/productos/{id_producto}	ADM	—	204	404
POST	/internas/proveedores/lote	ms-reportes (interno)	{filas: [{nit, razon_social, nombre_contacto, telefono, correo, direccion}]}	200 {creados, rechazados: [{fila, motivo}]} (RF16)	422 lista vacía

Eliminar un proveedor lo deja INACTIVO, porque las órdenes de compra anteriores lo siguen nombrando. Las asociaciones con productos sí se borran, porque no tienen historia. Al asociar un producto, ms-proveedores no consulta a ms-catalogo: el cliente web lo toma de la lista del catálogo.

4.7 ms-inventario (Express)

Tabla 9

Operaciones de ms-inventario

Método	Ruta interna	Roles	Entrada	Respuesta	Errores
GET	/bodegas	Todos; ms-reportes (interno)	?activa	200 listado	—
GET	/bodegas/{id}	Todos	—	200 bodega	404
POST	/bodegas	ADM	{codigo, nombre, ciudad, direccion}	201 bodega activa (RF05)	409 CODIGO_ DUPLICADO; 422
PUT	/bodegas/{id}	ADM	{nombre, ciudad, direccion}	200 bodega	404; 422
POST	/bodegas/{id}/inactivar	ADM	—	200; la bodega deja de recibir movimientos	404
POST	/bodegas/{id}/activar	ADM	—	200	404
GET	/saldos	Todos; ms-reportes (interno)	?id_producto&id_ bodega&pagina&por_ pagina	200 saldos por producto y bodega; con id_ producto, también el total de las dos bodegas (RF05)	—
GET	/saldos/bajo-umbral	ADM; ms-notificaciones y ms-reportes (interno)	—	200 [{id_producto, cantidad_total, cantidad_ minima}] (RF09)	—
GET	/umbrales	ADM	?id_producto&pagina& por_pagina	200 listado	—
PUT	/umbrales/{id_producto}	ADM	{cantidad_minima}	200 umbral; se crea si no existía (RF09)	404 producto inexistente; 422; 503 ms-catalogo no responde
POST	/movimientos	ADM, OPE; ms-compras (interno)	{tipo, id_producto, id_ bodega, cantidad, id_ lote?, lote?: {codigo_ lote, fecha_ vencimiento}, documento_referencia?, observacion?} + cabecera Idempotency-Key	201 movimiento con saldo_resultante (RF04, RF10); 200 con el movimiento original si la clave ya se usó (RNF17)	403 bodega ajena o tipo no permitido; 404 producto o bodega inexistente; 409 STOCK_ INSUFICIENTE; 422; 503 ms-catalogo no responde
GET	/movimientos	ADM, CON, OPE	?id_producto&id_ bodega&id_usuario& tipo&fecha_desde& fecha_hasta&pagina& por_pagina	200 kardex paginado, del más reciente al más antiguo (RF10, RF15)	422 filtros no válidos
GET	/movimientos/{id}	ADM, CON, OPE	—	200 movimiento	404
GET	/lotes	ADM, OPE; ms-notificaciones y ms-reportes (interno)	?id_producto&id_ bodega&vence_antes_ de&con_existencia& pagina&por_pagina	200 lotes, del que vence primero al último (RF17)	—
GET	/lotes/sugerencia-salida	ADM, OPE	?id_producto&id_ bodega&cantidad (obligatorios)	200 [{id_lote, codigo_ lote, fecha_vencimiento, cantidad_sugerida}], del más próximo a vencer al último (FEFO, RF17)	409 STOCK_ INSUFICIENTE; 422

(continuación)

Método	Ruta interna	Roles	Entrada	Respuesta	Errores
POST	/traslados	ADM, OPE	{id_producto, id_ bodega_origen, id_ bodega_destino, cantidad, id_lote?} + cabecera Idempotency-Key	201 traslado EN_ TRANSITO y movimiento TRASLADO_SALIDA en el origen (RF04)	403 bodega de origen ajena; 409 STOCK_ INSUFICIENTE; 422 misma bodega
POST	/traslados/{id}/recepcion	ADM, OPE	{observacion?} + cabecera Idempotency-Key	200 traslado RECIBIDO y movimiento TRASLADO_ ENTRADA en el destino	403 bodega de destino ajena; 404; 409 ya recibido
GET	/traslados	ADM, OPE	?estado&id_bodega& pagina&por_pagina	200 listado	—
POST	/conteos	ADM	{id_bodega, fecha_ programada, ids_ producto?}	201 conteo PROGRAMADO con sus líneas (RF18)	404 bodega inexistente; 422 fecha pasada
GET	/conteos	ADM, OPE	?id_bodega&estado& pagina&por_pagina	200 listado	—
GET	/conteos/{id}	ADM, OPE	—	200 conteo con sus líneas	404
PUT	/conteos/{id}/detalles	ADM, OPE	{detalles: [{id_producto, cantidad_contada}]}	200 conteo EN_ REVISION, con cantidad_sistema y diferencia en cada línea	404; 409 conteo ya aprobado; 422 producto fuera del conteo
POST	/conteos/{id}/aprobacion	ADM	— + cabecera Idempotency-Key	200 conteo APROBADO y un movimiento AJUSTE por cada diferencia (RF18)	404; 409 el conteo no está EN_ REVISION
POST	/internas/saldos/ carga-inicial	ms-reportes (interno)	{filas: [{id_producto, id_ bodega, cantidad, codigo_lote?, fecha_ vencimiento?}]} + cabecera Idempotency-Key	200 {creados, rechazados: [{fila, motivo}]}; cada fila crea un movimiento CARGA_INICIAL (RF16)	422 lista vacía

En POST /movimientos, los usuarios registran los tipos ENTRADA, SALIDA, AJUSTE y DEVOLUCION_CLIENTE, y ms-compras registra ENTRADA_COMPRA y DEVOLUCION_ PROVEEDOR. Los tipos TRASLADO_SALIDA y TRASLADO_ENTRADA los crean los traslados; CARGA_INICIAL, la carga masiva, y AJUSTE, también la aprobación de un conteo. La devolución de un cliente (RF13) es, por tanto, un movimiento más. En una salida sin id_lote, el servicio descuenta primero del lote más próximo a vencer (RF17).

Antes de registrar un movimiento o un umbral, el servicio confirma que el producto existe con GET /productos/{id} de ms-catalogo (llamada 1). Mientras registra un movimiento, el servicio bloquea el saldo para que dos movimientos simultáneos no lo dejen negativo. Si después del movimiento la suma de las dos bodegas queda por debajo del umbral, avisa a ms-notificaciones (llamada 3) sin esperar respuesta.

El operador de bodega solo puede registrar y consultar operaciones de la bodega que llega en la cabecera X-User-Bodega: en un traslado, la de origen al enviarlo y la de destino al recibirlo.

4.8 ms-compras (Express)

Tabla 10

Operaciones de ms-compras

Método	Ruta interna	Roles	Entrada	Respuesta	Errores
GET	/ordenes-compra	ADM, OPE; ms-reportes (interno)	?estado&id_proveedor& id_bodega_destino& fecha_desde&fecha_ hasta&pagina&por_ pagina	200 listado paginado	—
GET	/ordenes-compra/{id}	ADM, OPE	—	200 orden con sus líneas y recepciones	404
POST	/ordenes-compra	ADM	{id_proveedor, id_ bodega_destino, fecha_ esperada, detalles: [{id_ producto, cantidad_ pedida, precio_ unitario}]}	201 orden CREADA con numero y total (RF06)	422 orden sin líneas o con un producto repetido
PUT	/ordenes-compra/{id}	ADM	Igual que POST	200 orden	404; 409 ESTADO_ NO_EDITABLE (solo se edita en CREADA); 422
PATCH	/ordenes-compra/ {id}/estado	ADM	{estado: ENVIADA o ANULADA}	200 orden con su nuevo estado (RF06)	404; 409 TRANSICION_ INVALIDA
POST	/ordenes-compra/ {id}/recepciones	ADM, OPE	{lineas: [{id_detalle, cantidad, codigo_lote?, fecha_vencimiento?}]} + cabecera Idempotency-Key	201 recepciones; la orden pasa a RECIBIDA_PARCIAL o RECIBIDA (RF06)	403 bodega ajena; 409 la orden no está ENVIADA ni RECIBIDA_ PARCIAL; 422 cantidad mayor que la pendiente; 503 ms-inventario no responde
GET	/ordenes-compra/ {id}/recepciones	ADM, OPE	—	200 listado	404
POST	/devoluciones-proveedor	ADM, OPE	{id_proveedor, id_ producto, id_bodega, cantidad, motivo, id_ orden?, id_lote?} + cabecera Idempotency-Key	201 devolución con su movimiento DEVOLUCION_ PROVEEDOR (RF13)	403 bodega ajena; 409 STOCK_ INSUFICIENTE; 422; 503 ms-inventario no responde
GET	/devoluciones-proveedor	ADM, OPE	?id_proveedor&fecha_ desde&fecha_hasta& pagina&por_pagina	200 listado	—

Cambios de estado permitidos: CREADA → ENVIADA o ANULADA; ENVIADA → RECIBIDA_PARCIAL o RECIBIDA, de forma automática con cada recepción, o ANULADA si todavía no tiene recepciones; RECIBIDA_PARCIAL → RECIBIDA. RECIBIDA y ANULADA

son estados finales.

Por cada línea recibida, ms-compras registra en ms-inventario un movimiento ENTRADA_COMPRA con POST /movimientos (llamada 2) y solo guarda la recepción cuando inventario responde. Si una línea falla, la respuesta indica cuáles faltan, y al reintentar con la misma Idempotency-Key no se duplican las ya registradas (RNF17). Al terminar, avisa a ms-notificaciones que la orden se recibió (llamada 4). La devolución a proveedor sigue el mismo camino con un movimiento DEVOLUCION_PROVEEDOR.

ms-compras no consulta a ms-proveedores al crear una orden: el administrador elige el proveedor de la lista que el cliente web obtiene de GET /api/proveedores.

4.9 ms-notificaciones (Flask)

Tabla 11

Operaciones de ms-notificaciones

Método	Ruta interna	Roles	Entrada	Respuesta	Errores
GET	/notificaciones	Todos	?leida&tipo&pagina& por_pagina; se filtra por el rol del usuario	200 listado paginado (RF09, RF17)	—
POST	/notificaciones/{id}/leida	Todos	—	200 notificación marcada como leída	404 no existe o no es para su rol
GET	/preferencias/mia	Todos	—	200 preferencia; si no existe, recibir_correo es falso	—
PUT	/preferencias/mia	Todos	{correo?, recibir_correo}	200 preferencia	422 falta el correo
GET	/configuracion-alertas	ADM	—	200 {dias_anticipacion_ vencimiento, hora_ revision_diaria}	—
PUT	/configuracion-alertas	ADM	{dias_anticipacion_ vencimiento, hora_ revision_diaria}	200; se reprograma la revisión diaria (RF17)	422 valores no válidos
POST	/internas/notificaciones	ms-inventario y ms-compras (interno)	{tipo, mensaje, id_ producto?, id_bodega?, rol_destinatario?}	202 aceptada	422

Cada notificación va dirigida a un rol (ADMINISTRADOR por defecto) y cada usuario ve las de su rol. Si ya existe una alerta sin leer del mismo tipo y producto, no se crea otra. Al crearla, el servicio envía el correo a los usuarios de ese rol que activaron recibir_correo en su preferencia (llamada 9); si el envío falla, la alerta sigue visible en la interfaz.

La preferencia guarda el rol que llega en la cabecera X-User-Rol, para saber a quién

enviar cada correo. Cada día, a la hora configurada, el servicio consulta GET /lotes y GET /saldos/bajo-umbral de ms-inventario (llamada 5) y genera las alertas de vencimiento (RF17) y de stock bajo (RF09) que falten.

4.10 ms-reportes (Flask)

Tabla 12

Operaciones de ms-reportes

Método	Ruta interna	Roles	Entrada	Respuesta	Errores
GET	/reportes/existencias	ADM, CON	?id_bodega&id_ categoria&formato	200 existencias por producto y bodega (RF11)	503
GET	/reportes/valorizacion	ADM, CON	?id_bodega&formato	200 valor de cada producto y valor_total (RF11)	503
GET	/reportes/rotacion	ADM, CON	?fecha_desde&fecha_ hasta (obligatorios)& limite&formato	200 {mayor_rotacion, menor_rotacion} (RF11)	422 rango de fechas no válido; 503
GET	/reportes/kardex	ADM, CON	Los filtros de GET /movimientos y formato (csv o pdf)	200 archivo con el kardex filtrado (RF15)	503
GET	/indicadores	ADM, CON	?fecha_desde&fecha_ hasta	200 {valor_total, productos_bajo_umbral, productos_agotados, proximos_a_vencer, mayor_rotacion, ordenes_pendientes} (RF14)	503
POST	/importaciones	ADM	Archivo CSV o Excel y tipo (PRODUCTOS, PROVEEDORES o STOCK_INICIAL)	201 importación con filas_totales, filas_ validas, filas_con_error y estado (RF16)	422 formato no soportado o faltan columnas
GET	/importaciones	ADM	?tipo&pagina&por_ pagina	200 listado	—
GET	/importaciones/{id}	ADM	—	200 importación con sus errores por fila	404

El parámetro formato admite json (por defecto), csv o pdf; con csv o pdf, la respuesta es un archivo para descargar (RF16). Los reportes y los indicadores se calculan en el momento con datos de ms-inventario, ms-catalogo y ms-compras (llamada 6) y no se guardan. La valorización multiplica la cantidad en existencia por el precio registrado del producto, y la rotación ordena los productos según sus salidas en el periodo consultado (RF11).

En una importación, el servicio valida cada fila, convierte los códigos de producto, categoría y bodega en sus identificadores y envía las filas válidas al servicio dueño del dato (llamada

7). Las filas rechazadas quedan en ErrorImportacion con su número de fila, el campo y el mensaje.

La Tabla 13 muestra las columnas que debe tener cada tipo de archivo de la carga masiva y el servicio que recibe sus filas válidas.

Tabla 13

Columnas de los archivos de carga masiva

Tipo	**Columnas del archivo**	**Servicio que recibe las filas**
PRODUCTOS	codigo, nombre, descripcion, categoria, unidad_medida, precio	ms-catalogo
PROVEEDORES	nit, razon_social, nombre_contacto, telefono, correo, direccion	ms-proveedores
STOCK_INICIAL	codigo_producto, codigo_bodega, cantidad, codigo_lote, fecha_vencimiento	ms-inventario

Nota. La categoría, el producto y la bodega se escriben por su código o nombre en el archivo, y ms-reportes los convierte en sus identificadores antes de enviar las filas (RF16). Elaboración propia.

5. Bitácora de uso de IA

La bitácora sigue la plantilla de la sección 7 del enunciado: un encabezado con los datos de la entrega, una fila por cada interacción relevante con una herramienta de IA y una reflexión final de la fase (RNF16).

5.1 Datos de la bitácora

Tabla 14

Encabezado de la bitácora de uso de IA

Campo	Valor
Proyecto	Sistema de Gestión de Inventarios – Distribuidora El Montañero S.A.S.
Equipo / Integrantes	Luis Alejandro Saraza Marín, Brayan David Collazos Escobedo, Mateo Falla Muñoz
Entrega N.°	2
Periodo cubierto	19/10/2026 – 26/10/2026

5.2 Registro de interacciones

La Tabla 15 registra, en orden cronológico, las cuatro interacciones de la fase con herramientas de IA.

Tabla 15

Registro de interacciones con herramientas de IA en la Entrega 2

Fecha	Integrante	Herramienta de IA	Tarea / actividad apoyada	Prompt utilizado (resumen)	Nivel de intervención humana	Resultado / aprendizaje
23/10/2026	Brayan David Collazos Escobedo	Claude (Claude Code)	Creación y mejora del diagrama de arquitectura de microservicios.	A partir de la Entrega 1 y del enunciado, generar el diagrama de arquitectura (componentes, comunicación, API Gateway propio y descubrimiento de servicios).	Bajo	Al ser la IA una herramienta estructurada, y con la potencia actual de la misma, a la primera arrojó un diagrama de arquitectura viable. Para este tipo de tareas la IA funciona bien, siempre que se le ingrese una buena instrucción.
24/10/2026	Luis Alejandro Saraza Marín	Claude (Claude Code)	Creación de los modelos de datos (modelo entidad-relación).	A partir de la Entrega 1 corregida y del enunciado, generar un modelo entidad-relación en notación de Chen por cada componente con base de datos.	Medio	Si bien la IA es capaz de generar todos los modelos de una vez, fue necesario pedirle los diagramas uno a uno, para mejor claridad y calidad. Tocó pedirle varias veces que los generara bien.
25/10/2026	Mateo Falla Muñoz	Claude (Claude Code)	Creación de los contratos de API.	A partir de la Entrega 1 corregida y del enunciado, así como de la arquitectura del sistema y de las bases de datos, generar los contratos de API.	Alto	Para una tarea tan grande, la IA no fue capaz de limitar el alcance sin una base concreta sobre la cual trabajar. Por esto se tuvieron que hacer múltiples intentos y correcciones para obtener el producto actual.
25/10/2026	Mateo Falla Muñoz	Claude (Claude Code)	Organización final del documento: retiro de la tabla de contenido, rediseño de la bitácora, verificación frente a los requerimientos y conversión a LaTeX.	Quitar el apartado de índices; organizar y mejorar el diseño de la bitácora de IA; comparar la validez de la solución con los requerimientos de la segunda entrega y su concordancia con la primera, y rediseñar el documento en formato LaTeX.	Medio	La IA retiró la tabla de contenido, organizó la bitácora según la plantilla del enunciado y pasó el documento a LaTeX. La comparación confirmó que la arquitectura, los modelos de datos y los contratos cubren los requerimientos RF01 a RF18 y concuerdan con la primera entrega, y señaló los entregables del enunciado que esta fase no desarrolla, para confirmarlos con el docente. Se aprendió que revisar el documento contra el enunciado antes de entregarlo evita dejar vacíos.

Nota. El nivel de intervención humana (Alto, Medio o Bajo) indica cuánto modificó el equipo lo que generó la IA. Elaboración propia.

5.3 Reflexión final de la fase

En esta fase usamos la IA como asistente de diseño. Le delegamos el dibujo de los diagramas a partir de las entidades de la primera entrega, la redacción de los contratos de API y la organización del documento, porque son productos extensos que siguen convenciones conocidas. Sin embargo, la IA no fue capaz de definir bien todo el documento: fue necesario un nivel de intervención considerable, puesto que en varias respuestas había errores. Para verificar lo generado, revisamos cada diagrama y cada tabla frente a la primera entrega y al enunciado, y pedimos correcciones hasta que coincidieran. Esto demuestra que la IA es una gran herramienta de apoyo, pero todavía necesita filtros humanos para garantizar el correcto cumplimiento de las tareas asignadas, porque a veces simplemente hace lo que entiende, cosa que no siempre está en concordancia con lo que el usuario tiene en mente.

6. Conclusiones

El diseño concreta la arquitectura de la primera entrega sin cambiar sus límites: seis microservicios y un API Gateway propio en Laravel que concentra la entrada, la identidad y el control de acceso. Usar solo Laravel, Express y Flask, con MySQL y PostgreSQL, mantiene el proyecto dentro de las tecnologías vistas en clase.

La comunicación queda definida en detalle: nueve llamadas REST entre servicios, separadas entre las que esperan respuesta y los avisos que no detienen la operación, con tiempos máximos de espera, reintentos controlados y claves de idempotencia para no duplicar movimientos.

Los modelos entidad-relación y los contratos en tablas dan a cada servicio una base de datos propia y una interfaz explícita, sin claves foráneas entre servicios, y la matriz de permisos traduce los cuatro roles en reglas que aplica el Gateway. Con esto, la siguiente entrega puede empezar la implementación de cada servicio sobre interfaces ya acordadas entre los integrantes.

Referencias

Chen, P. P.-S. (1976). The entity-relationship model—Toward a unified view of data. *ACM Transactions on Database Systems, 1*(1), 9–36. https://doi.org/10.1145/320434.320440

Jones, M., Bradley, J. y Sakimura, N. (2015). *JSON Web Token (JWT)* (RFC 7519). Internet Engineering Task Force. https://doi.org/10.17487/RFC7519

Kubernetes. (2025, 29 de julio). *DNS for Services and Pods*. Kubernetes Documentation. https://kubernetes.io/docs/concepts/services-networking/dns-pod-service/

Nottingham, M. y Fielding, R. (2012). *Additional HTTP status codes* (RFC 6585). Internet Engineering Task Force. https://doi.org/10.17487/RFC6585

Richardson, C. (s. f.-a). *Pattern: API gateway / Backends for frontends*. Microservices.io. Recuperado el 25 de octubre de 2026, de https://microservices.io/patterns/apigateway.html

Richardson, C. (s. f.-b). *Pattern: Database per service*. Microservices.io. Recuperado el 25 de octubre de 2026, de https://microservices.io/patterns/data/database-per-service.html

Saraza Marín, L. A., Collazos Escobedo, B. D. y Falla Muñoz, M. (2026). *Sistema de gestión de inventarios basado en arquitectura de microservicios. Entrega 1: Investigación y fundamentación* [Trabajo de curso, Universidad Nacional de Colombia, Sede Manizales]. https://github.com/Mat-FM/Sistemas_Gestor_Inventario